

LeapMotion Gesture Controller

Touchless Human–Computer Interaction for macOS

Aysun Suleymani

Platform: macOS · Language: C++17 · SDK: Ultraleap LeapC

Table of Contents

1. Introduction.....	3
1.1 The Ultraleap Controller.....	3
1.2 Project Overview	3
2. Motivation.....	4
3. Requirements and Prerequisites.....	4
3.1 Hardware Requirements	4
3.2 Software Requirements.....	4
3.3 System Permission.....	5
4. Design, Architecture and Implementation.....	6
4.1 Architectural Overview	6
4.2 Module Descriptions.....	6
4.3 Pose Classification and Stability Filtering.....	7
4.4 Gesture Recognition Strategies.....	7
Velocity Thresholding — taps, scroll, swipe.....	7
Delta Accumulation — zoom, rotate	7
Dual-Threshold Hysteresis — drag	8
Hold-to-Continue — zoom extension.....	8
4.5 Coordinate Mapping and Event Injection.....	8
4.6 Configuration.....	8
5. Installation and Usage.....	10
5.1 Installation Steps.....	10
5.2 Gesture Reference.....	10
5.3 Debug Log Reference	11
5.4 Tuning.....	11
6. Limitations and Future Work.....	11
6.1 Current Limitations.....	11
6.2 Potential Extensions.....	12

1. Introduction

1.1 The Ultraleap Controller

The Ultraleap controller — originally launched as the Leap Motion — is a compact optical sensor designed to track the three-dimensional position of a user's hands in real time. Mounted on a desk and facing upward, the device uses two infrared cameras and a set of infrared LEDs to capture stereo imagery of the space above it. Ultraleap's on-device processing pipeline reconstructs a full skeletal model of each hand from this imagery, providing per-frame data that includes:

- **Palm position and velocity** (millimetres in the device coordinate frame).
- **Individual fingertip positions** for all five digits.
- **Per-finger extension state** (straight or curled).
- **Derived metrics:** pinch strength, thumb–index distance, and grab (fist) strength.

This data is delivered through the **Ultraleap Hand Tracking Service**, a background process that exposes the LeapC C API to client applications. The service runs at approximately **110 frames per second**, ensuring low-latency responsiveness for interactive use. The controller supports both Desktop mode (sensor on a flat surface, facing up) and HMD (head-mounted) mode; this project uses Desktop mode exclusively.

1.2 Project Overview

LeapMotion Gesture Controller is a native macOS application that translates Ultraleap hand-tracking data into standard system-level input events. By synthesising the same event types that a physical mouse and trackpad produce — cursor movement, mouse clicks, scroll-wheel events, multi-touch swipes, and keyboard shortcuts — the application enables touchless control of any macOS application without those applications requiring any modification.

The gesture set covers the complete vocabulary of everyday pointer interaction:

Gesture	Input Produced
Index finger extended	Cursor movement
Index finger tap	Tap to Click (Left click)
Two-finger tap	Right click
Two-finger push/pull	Scroll
Two-finger horizontal flick	Swipe navigation
Thumb–index pinch	Zoom in / zoom out
Open flat hand, wrist rotation	Image rotation
Closed fist, translate, open	Drag and drop

2. Motivation

This project was driven by a combination of practical objectives and technical interest:

- **Touchless, spatial interaction.** Conventional pointing devices require physical contact with a surface. Hands-in-the-air control is well suited to scenarios where this is impractical — presentations, controlled environments, accessibility needs, or simply the ergonomic preference for a more natural, three-dimensional interaction style.
- **Complete input replacement, not a demo.** Most hand-tracking prototypes demonstrate a single capability in isolation. The objective here was to produce a system that covers the full range of everyday pointer gestures so that a user can work without a trackpad at all.
- **Accessibility.** Fine motor control required by a trackpad can be a barrier for some users. A gesture interface calibrated to gross hand movements offers an alternative input path.
- **Depth-exclusive interactions.** A flat surface cannot express gestures that use the third dimension. Pinch-based zoom, wrist-rotation-based image rotation, and depth-of-field navigation are gestures that have no natural analogue on a trackpad and are worth exploring in their own right.
- **End-to-end systems engineering.** Building the complete pipeline — from raw noisy sensor data through pose classification and gesture recognition to OS event injection — is a rich exercise in real-time signal processing, finite state machine design, and platform integration.

3. Requirements and Prerequisites

3.1 Hardware Requirements

Component	Specification
Sensor	Ultraleap / Leap Motion Controller (USB, any generation compatible with the Gemini V5 service).
Host machine	Apple Mac with a free USB-A or USB-C port (USB-C adapter required for original Leap Motion on newer Macs).
Sensor placement	Flat on the desk, cameras facing upward (Desktop mode). Optimal working height: 25–60 cm above the sensor.

3.2 Software Requirements

Requirement	Version / Notes
macOS	12.0 (Monterey) or later.
Ultraleap Hand Tracking Service	Gemini V5 or later. Must be running before the application is launched.
CMake	3.16 or later.
Xcode Command Line Tools	xcode-select --install. Provides clang with C++17 support.
CoreGraphics & ApplicationServices	Part of the macOS SDK; no separate installation required.

Requirement	Version / Notes
Ultraleap SDK headers	Bundled in the repository under external/ultraleap/. No separate SDK download is required to compile.

3.3 System Permission

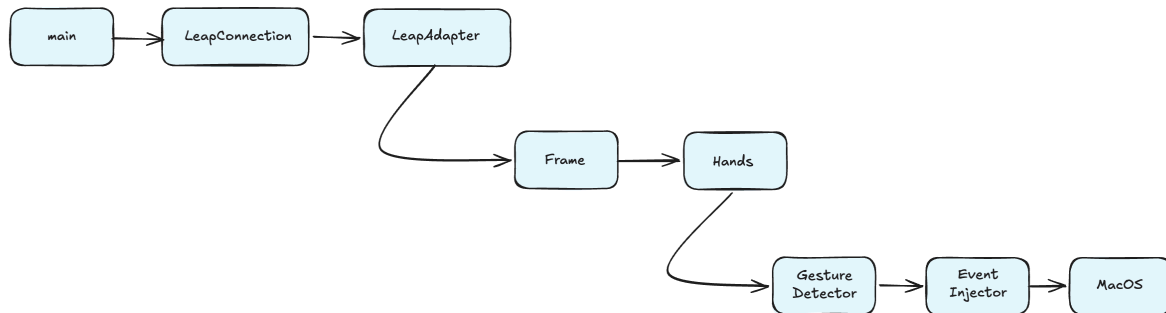
macOS requires applications that synthesise input events to hold **Accessibility permission**. This is granted once in **System Settings** → **Privacy & Security** → **Accessibility**. Without it, the application will launch but will be unable to move the cursor or post any events.

NOTE	The Ultraleap SDK headers and import library ship inside the repository under external/ultraleap/, so no separate SDK download is necessary to compile the project. Only the Ultraleap Hand Tracking Service runtime must be installed separately.
-------------	--

4. Design, Architecture and Implementation

4.1 Architectural Overview

The application follows a linear pipeline architecture. Each iteration of the main loop ingests the most recent tracking frame and produces at most one system event. The data path is:



This layered design enforces a strict separation of concerns: only the input layer (LeapConnection, LeapAdapter) has knowledge of the Ultraleap SDK, and only the output layer (EventInjector) has knowledge of the macOS CoreGraphics API. Every module in between operates on typed, SDK-agnostic data structures. This makes the system straightforward to reason about and straightforward to extend — replacing the sensor or targeting a different OS would require changes only at the respective ends of the pipeline.

4.2 Module Descriptions

Module	Responsibility
LeapConnection	RAII wrapper around the LeapC connection handle. Opens the connection, selects Desktop tracking mode, and guarantees the handle is released on destruction.
LeapAdapter	Polls the LeapC message queue each iteration. Drains the entire queue and retains only the most recent tracking frame, ensuring end-to-end latency stays at approximately one frame (~9 ms at 110 fps) regardless of processing time.
Frame	Immutable snapshot of one polling cycle. Wraps a LEAP_TRACKING_EVENT and exposes a typed list of Hand objects.
Hand	Read-only per-frame view of one hand. Exposes pinch strength, thumb–index distance, grab strength, palm position and velocity, fingertip positions, and per-finger extension flags. No LeapC types are exposed through the interface.
GestureDetector	The recognition core. Classifies each hand into a Pose , applies a stability filter, dispatches to the active pose handler, and runs the gesture-specific detection logic.
EventInjector	Constructs and posts CGEvent objects (cursor, clicks, scroll-wheel, multi-touch) and keyboard shortcuts for zoom and rotate. The sole interface to the operating system.

Module	Responsibility
Config	A single namespace of compile-time constants. Every threshold, cooldown, and sensitivity value is defined here with explanatory comments.
Vector3	Minimal 3-D vector type providing subtraction, dot product, Euclidean length, and the planar angle helpers required by the rotation gesture.

4.3 Pose Classification and Stability Filtering

Raw tracking data contains per-frame noise, transient occlusions, and the ambiguous hand shapes that arise during gesture transitions. The first processing stage therefore reduces a hand to one of six discrete **Poses**:

Pose	Trigger Condition	Gesture Family
None	No fingers clearly extended; hand idle.	No action.
Fist	Grab strength above threshold.	Drag and drop.
OneFinger	Index finger extended only.	Cursor movement, left click.
TwoFinger	Index and middle fingers extended.	Scroll, swipe, right click, smart zoom.
Pinch	Pinch strength above threshold.	Zoom in / zoom out.
OpenHand	All four fingers extended.	Image rotation.

A candidate pose must persist for POSE_STABILITY_FRAMES consecutive frames before it is committed. This debounce window (default: 6 frames, ~55 ms) absorbs transient misclassifications without introducing perceptible latency for deliberate gestures. Each pose owns an exclusive set of gestures, which eliminates cross-gesture interference — most importantly, zoom (**Pinch**) and rotate (**OpenHand**) can never interfere with each other.

4.4 Gesture Recognition Strategies

The application employs four distinct recognition techniques, selected according to the temporal structure of each gesture:

Velocity Thresholding — taps, scroll, swipe

A downward fingertip velocity crossing TAP_VELOCITY_THRESHOLD triggers a click. Sustained forward/backward palm motion produces scroll events scaled to palm speed. A horizontal palm velocity above SWIPE_VELOCITY_THRESHOLD (450 mm/s default), sustained for SWIPE_SUSTAIN_FRAMES frames, triggers a swipe. Per-gesture cooldown counters prevent a single motion from firing multiple events.

Delta Accumulation — zoom, rotate

Small per-frame changes are summed into an accumulator. When the accumulator exceeds a step threshold (ZOOM_STEP_MM for zoom, ROTATE_STEP_DEG for rotate), one discrete action fires and the accumulator resets. Per-frame contributions are capped at ZOOM_FRAME_CAP to prevent a single jittery frame from triggering a step. For rotation, the angle is measured across the index-to-pinky fingertip span

(~80 mm), which is significantly wider and more stable than the thumb-to-index gap used by earlier designs.

Dual-Threshold Hysteresis — drag

Drag onset requires grab strength to exceed `FIST_GRAB_MIN` for a sustained period; release requires it to fall below a lower threshold for a sustained period. The asymmetric thresholds prevent tremor in a closed fist from causing unintended drop events.

Hold-to-Continue — zoom extension

Finger travel is mechanically limited (~40 mm of pinch travel). Without additional logic, zoom would stop when the pinch reaches its extremes. Once an active zoom direction has been established in the current pinch session (tracked by `zoomedInThisPinch` / `zoomedOutThisPinch`) and the pinch is held at or beyond its travel limit, the application continues emitting zoom steps at a rate of one per `ZOOM_CONTINUE_FRAMES` frames. This continues until the pinch is partially released, providing effectively unlimited zoom range.

4.5 Coordinate Mapping and Event Injection

Fingertip and palm positions are converted from the sensor's millimetre coordinate frame to absolute screen pixels via a linear mapping calibrated to the working volume above the sensor. A first-order low-pass filter is applied to the cursor position to attenuate high-frequency jitter while preserving intentional movement.

`EventInjector` synthesises events using the CoreGraphics `CGEvent` API, posting them to `kCGHIDEventTap` so they are indistinguishable from physical device input. Zoom is mapped to Command + / Command -; rotation to Command R / Command L (the Preview image-rotation shortcuts). Scroll uses `CGEventCreateScrollWheelEvent2` with pixel-granularity deltas for smooth behaviour. Multi-touch swipe and smart-zoom are synthesised as trackpad gestures using `CGEventCreateCopy` with appropriate phase and modifier flags.

4.6 Configuration

All tunable parameters are defined as `constexpr` constants in `src/core/Config.h`. This includes pose-classifier thresholds, gesture sensitivity values, cooldown durations, and smoothing factors. Each constant carries an inline comment describing its effect and a reasonable adjustment range. No rebuild of recognition logic is required to adjust behaviour — only `Config.h` need be changed.

Constant	Controls
<code>POSE_STABILITY_FRAMES</code>	Frames a candidate pose must persist before committing (debounce).
<code>ZOOM_STEP_MM</code>	Pinch-distance change (mm) required to emit one zoom step.
<code>ZOOM_CONTINUE_FRAMES</code>	Frame interval between hold-to-continue zoom steps.
<code>SWIPE_VELOCITY_THRESHOLD</code>	Minimum palm speed (mm/s) to trigger a swipe.
<code>ROTATE_STEP_DEG</code>	Wrist rotation (degrees) required per image rotation step.

Constant	Controls
ROTATE_MIN_SPAN	Minimum index-to-pinky span (mm) for open-hand pose to engage.
FIST_GRAB_MIN	Grab strength required to begin a drag.

5. Installation and Usage

5.1 Installation Steps

Step 1 — Install the Ultraleap Hand Tracking Service. Download and install the Gemini V5 (or later) runtime from the Ultraleap developer portal. Open the Ultraleap control panel and verify the sensor is detected and streaming.

Step 2 — Install build tools. If the Xcode Command Line Tools are not already present:

```
xcode-select --install
```

Step 3 — Build the project. From the project root directory:

```
mkdir build && cd build
cmake ..
make
```

Step 4 — Grant Accessibility permission. Launch the binary once:

```
./leap
```

macOS will display a prompt. Navigate to **System Settings** → **Privacy & Security** → **Accessibility** and enable the toggle for the leap binary. Relaunch the binary to begin operation.

NOTE	The application prints live diagnostic output to the terminal — the current hand pose and a [fire] line for every gesture that executes. This output is the primary tool for verifying correct operation during testing.
-------------	--

5.2 Gesture Reference

Gesture	Hand Configuration	Application
Cursor movement	Index finger extended; all others curled.	System-wide
Left click	Index extended; brief downward tap.	System-wide
Right click	Index and middle extended; brief downward tap.	System-wide
Scroll	Index and middle extended; push hand away (up) or pull toward you (down).	System-wide
Swipe	Index and middle extended; rapid horizontal flick (≥ 450 mm/s).	Safari, Photos, etc.
Zoom in	Pinch; spread thumb and index apart. Hold spread to continue.	Safari, Preview
Zoom out	Pinch; bring thumb and index together. Hold pinched to continue.	Safari, Preview
Rotate	All fingers extended, hand flat; rotate wrist left or right.	Preview

Gesture	Hand Configuration	Application
Drag	Close hand into a fist over the target; translate hand to destination; open hand.	Finder, Desktop

5.3 Debug Log Reference

When Config::DEBUG_GESTURES is enabled (the default), every gesture emits a structured log line:

Gesture	Log Output
Left click	[fire] LEFT CLICK at (X,Y)
Right click	[fire] RIGHT CLICK
Scroll	[fire] SCROLL vY=...
Swipe	[fire] swipe LEFT (vx=...) / swipe RIGHT (vx=...)
Zoom in/out	[fire] ZOOM IN / ZOOM OUT
Zoom (hold)	[fire] ZOOM IN (hold at ceiling) / ZOOM OUT (hold at floor)
Rotate	[fire] ROTATE + / ROTATE -
Smart zoom	[fire] SMART ZOOM
Drag / drop	[fire] DRAG START at (X,Y) ... DROP at (X,Y)

5.4 Tuning

All sensitivity and threshold values are configurable in src/core/Config.h without modifying recognition logic. Recompile after any change. Common adjustments:

Symptom	Constant to adjust	Direction
Zoom steps fire too often / too rarely.	ZOOM_STEP_MM	Increase to require more pinch travel per step; decrease for greater sensitivity.
Swipes trigger unintentionally.	SWIPE_VELOCITY_THRESHOLD	Increase to require a faster flick.
Rotation is too sensitive.	ROTATE_STEP_DEG	Increase to require a larger wrist turn per step.
Pose changes are too slow or too jittery.	POSE_STABILITY_FRAMES	Decrease for faster switching; increase for more stability.
Hold-to-zoom continues too quickly.	ZOOM_CONTINUE_FRAMES	Increase to slow the repeat rate.

6. Limitations and Future Work

6.1 Current Limitations

Limitation	Detail
Finite pinch travel	Pinch-based zoom is bounded by the physical distance between thumb and index tip (~40 mm). The hold-to-continue mechanism mitigates this, but an alternative using palm vertical displacement would offer unlimited range.
Application-dependent shortcuts	Zoom and rotate are emitted as keyboard shortcuts (Command + / – and Command R / L). Their effect depends on the focused application; some applications assign these keys differently.
Single active gesture	The pose-stability model commits to one hand shape at a time. Simultaneous multi-gesture input is architecturally out of scope for the current design.
Tracking quality dependency	Gesture reliability degrades in poor lighting, at the boundary of the sensor's field of view, or when hands are partially occluded. The Config.h thresholds can be adjusted to trade sensitivity against false-positive rate.

6.2 Potential Extensions

- **Palm-depth zoom.** Mapping vertical palm translation to zoom would provide unlimited range without the hold-to-continue workaround.
- **Two-handed gestures.** The per-hand state architecture already tracks multiple hands independently; bindings for two-handed interactions require only new gesture handlers.
- **On-screen pose indicator.** A small HUD overlay displaying the currently recognised pose would reduce the learning curve.
- **Per-application profiles.** Different Config.h constants could be loaded dynamically based on the frontmost application, allowing zoom sensitivity optimised for Safari and scroll sensitivity optimised for a code editor to coexist.
- **API documentation generation.** Every class and method carries Doxygen-compatible comments. Running doxygen Doxyfile from the project root produces a fully cross-referenced HTML API reference in docs/html/.